Практическая работа 5. Конфигурационное управление

Файл – это виртуальная информационная единица, у которой может быть много версий. *Версия файла* – отражает текущее его состояние и одна версия от другой может отличаться несколькими строчками кода или полностью обновленным содержанием.

В программных проектах необходима специальная деятельность по поддержанию файловых активов проекта в актуальном состоянии, которая называется *конфигурационным управлением*.

В конфигурационном управлении выделяют две основные задачи – *управление версиями* и *управление сборками*. Первая отвечает за управление версиями файлов и выполняется в проекте на основе специальных программных пакетов – *средств версионного контроля*. Существует большое количество таких средств – Microsoft Visual SourceSafe, IBM ClearCase, cvn, subversion и др.

Управление сборками – это автоматизированный процесс трансформации исходных текстов ПО в пакет исполняемых модулей, учитывающий многочисленные настройки проекта, компиляции, и интегрируемый с процессом автоматического тестирования. Эта процедура является мощным средством интеграции проекта, основой итеративной разработки.

Единицы конфигурационного управления

Конфигурационное управление имеет дело с меняющимися в процессе проектирования наборами файлов, которые называют *единицами конфигурационного управления*, например:

- пользовательская документация;
- проектная документация;
- исходные тексты ПО;
- пакеты тестов;
- инсталляционные пакеты ПО;
- тестовые отчеты.

Единицы конфигурационного управления характеризуются следующим:

1. *Структура* – набор файлов. Например, пользовательская документация в html должна включать индекс-файл и набор

html-файлов, а также набор вынесенных картинок (gif или jpeg-файлы). Эта структура должна быть хорошо определена и отслеживаться при конфигурационном управлении – что все файлы не потеряны и присутствуют, имеют одинаковую версию, корректные ссылки друг на друга и т.д.

2. *Ответственное лицо* и, возможно, группу тех, кто их разрабатывает, а также более широкую и менее ответственную группу тех, кто пользуется этой информацией. Например, определенной программной компонентой могут в проекте пользоваться многие разработчики, но отвечать за ее разработку, исправление ошибок должен кто-то один.
3. *Практика конфигурационного управления* – кто и в каком режиме, а также в какое место выкладывает новую версию элемента конфигурационного управления в средство управления версиями, правила именования и комментирования элемента в этой версии, дальнейшие манипуляции с ним. Более высокоуровневые правила, связанные, например, с правилами изменения тестов и тестовых пакетов при изменении кода.
4. *Автоматическая процедура контроля целостности элемента* – например, сборка для исходных текстов программ. Это свойство не относится к элементам конфигурационного управления, например, может не быть у документации, тестовых пакетов.

Элементы конфигурационного управления могут образовывать иерархию. Пример представлен на рис. 1.

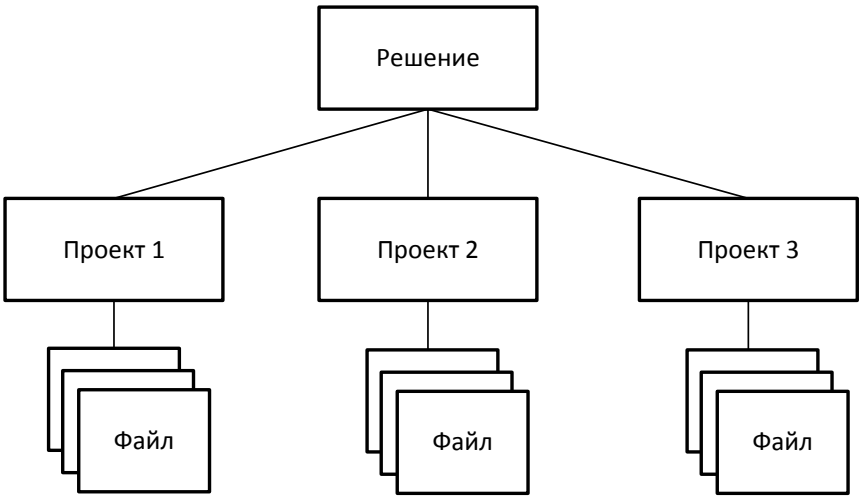


Рисунок 1 *– Элементы конфигурационного управления*

Управление версиями

Управление версиями файлов. В процессе разработки ПО создается большое количество файлов. Файлы могут одновременно использоваться различными членами команды. Для согласованной работы над проектом файлы должны быть актуальными. Поэтому необходима постоянная работа по управлению версиями файлов.

Управление версиями составных конфигурационных объектов. Понятие «ветки» проекта. Одновременно может существовать несколько версий системы – и для разных заказчиков, и в одном проекте одного заказчика, но как разные наборы исходных текстов. Для этого при управлении версиями используют разные *ветки*.

Каждая ветка содержит полный образ исходного кода и других артефактов, находящихся в системе контроля версий. Каждая ветвь может развиваться независимо, а может в определенных точках интегрироваться с другими ветвями. В процессе интеграции изменения, произведенные в одной из ветвей, полуавтоматически переносятся в другую. В качестве примера можно рассмотреть следующую структуру разделения проекта на ветки:

- master – ветвь, соответствующая основному направлению развития проекта. По мере созревания именно от этой ветви отходят ветви готовящихся релизов;
- v1.0 – ветвь, соответствующая выпущенному релизу. Внесение изменений в такие ветви запрещено, и они хранят образ кода системы на момент выпуска релиза;
- fix v1.0.1 – ветвь, соответствующая выпущенному пакету исправлений к определенной версии. Подобные ветви ответвляются от исходной версии, а не от основной ветви и замораживаются сразу после выхода пакета исправлений;
- upcoming (V1.1) – ветвь, соответствующая релизу, готовящемуся к выпуску и находящемуся в стадии стабилизации. Для таких ветвей, как правило, действуют более строгие правила и работа в них ведется более формально;
- experiment1 – ветвь, созданная для проверки некоторого технического решения, перехода на новую технологию, или

внесения большого пакета изменений, потенциально нарушающих работоспособность кода на длительное время. Такие ветви, как правило, делаются доступными только для определенного круга разработчиков и удаляются по завершении работ после интеграции с основной веткой.

Системы контроля версий

Система контроля версий (СКВ) – это система, регистрирующая изменения в одном или нескольких файлах, с тем чтобы в дальнейшем была возможность вернуться к определённым старым версиям этих файлов. В СКВ целесообразно помещать исходные коды программ, но под версионный контроль можно поместить файлы практически любого типа.

СКВ даёт возможность возвращать отдельные файлы к прежнему виду, возвращать к прежнему состоянию весь проект, просматривать происходящие со временем изменения, определять, кто последним вносил изменения во внезапно переставший работать модуль, кто и когда внёс в код какую-то ошибку, и многое другое. СКВ позволяет восстановить испорченные или потерянные файлы.

СКВ позволяют контролировать версии в локальной базе данных, в которой хранятся все изменения нужных файлов (рис. 2).



Рисунок 2 *– Схема локальной СКВ*

Централизованные системы контроля версий (ЦСКВ) предназначены для совместного использования кода командой разработчиков. В таких системах, например, CVS, Subversion и Perforce, есть центральный сервер, на котором хранятся все файлы под версионным контролем, и ряд клиентов, которые получают копии файлов из него (рис. 3).

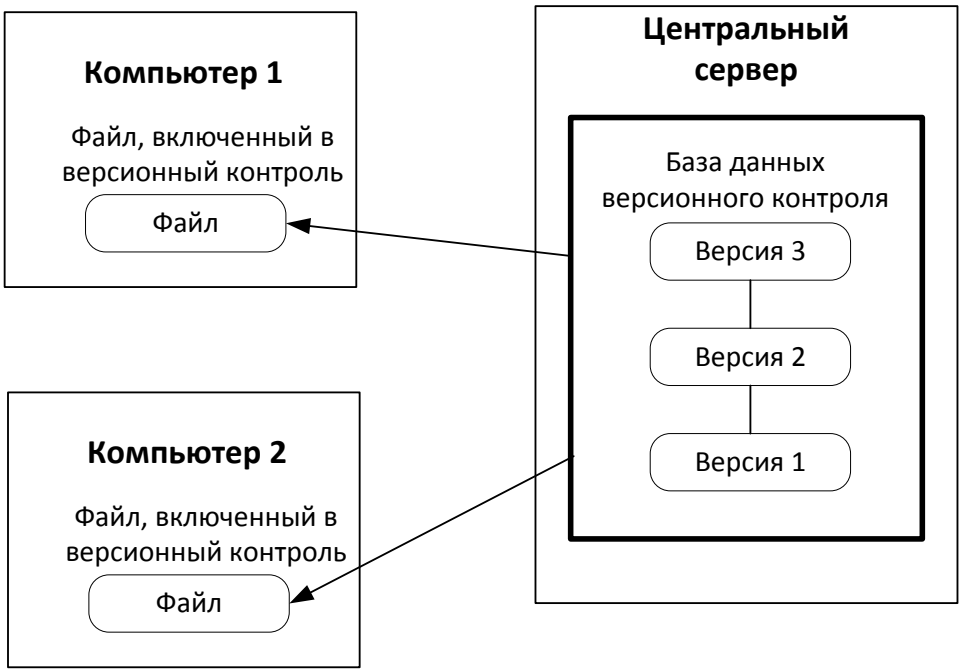


Рисунок 3 *– Схема централизованного контроля версий*

Система централизованного контроля версий имеет ряд преимуществ по отношению к локальной СКВ:

- доступ к СКВ нескольких членов команды разработчиков;
- прозрачность состояния проекта для заинтересованных лиц;
- фиксация и мониторинг всех изменений файлов проекта для всех членов команды разработчиков.

В то же время ЦСКВ имеет существанный недостаток – централизованный сервер является критичным ресурсом, и при выходе его из строя все разработчики теряют возможность доступа к информации по версиям файлов проекта.

В *распределённых системах контроля версий* (РСКВ), таких системах как: Git, Mercurial, Bazaar или Darcs клиенты не просто выгружают последние версии файлов, а полностью копируют весь репозиторий. Поэтому в случае, когда выходит из строя сервер, через который шла работа, любой клиентский репо-

зиторий может быть скопирован обратно на сервер, чтобы восстановить базу данных. Каждый раз, когда клиент забирает свежую версию файлов, он создаёт себе полную копию всех данных (рисунок 4).

В Visual Studio Team Foundation Server, начиная с версии 20 13 и Visual Studio OnLine, имеется возможность использовать две системы управления версиями: TFVS и Git.

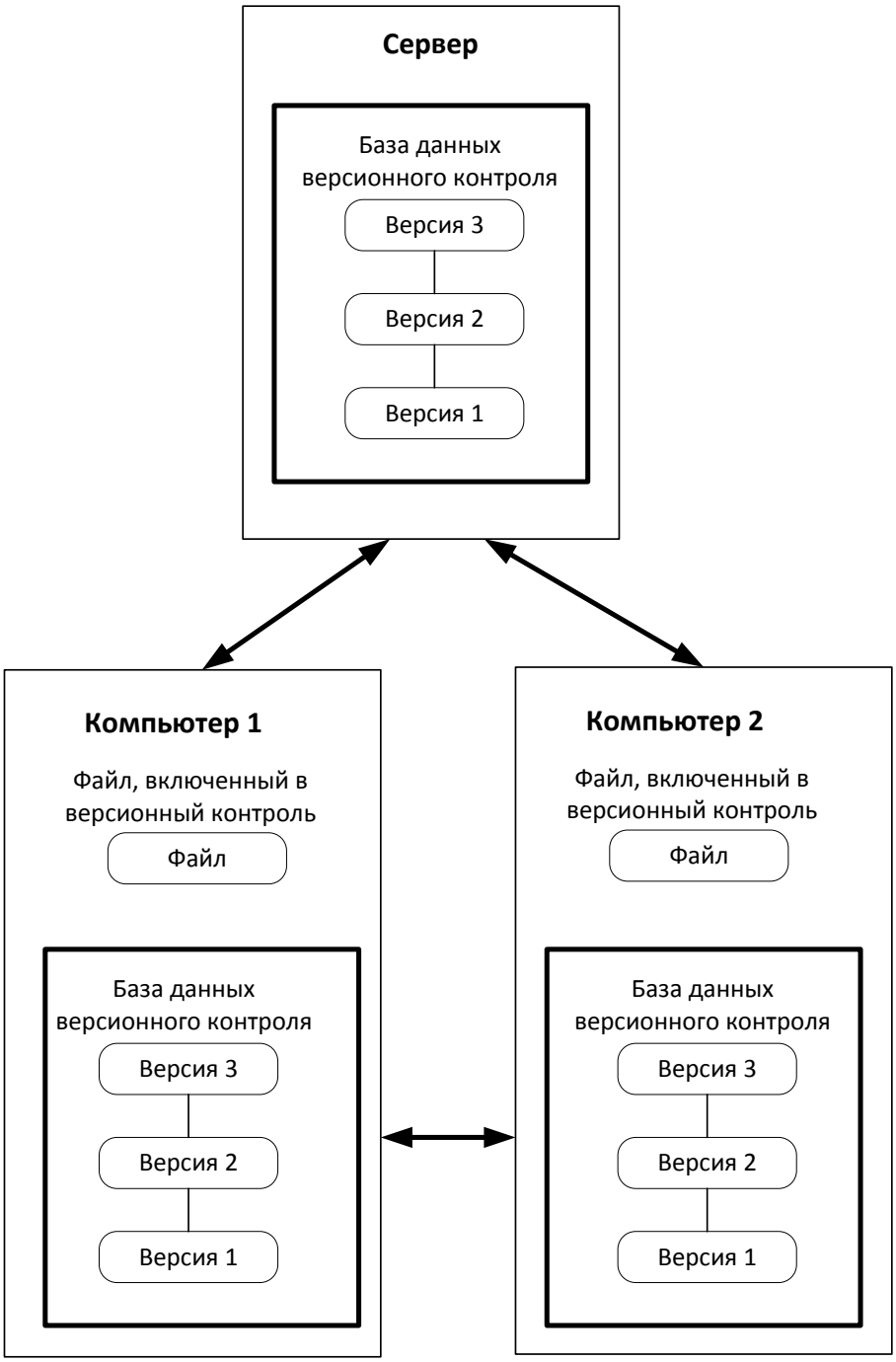


Рисунок 4 *– Схема распределённой системы контроля версий*

TFVS представляет решения по контролю версий исходного кода корпоративного уровня: отслеживание изменений исходного кода, атомарные возвраты, ветвление, объединение и отложенные изменения исходного кода, политики и защиту кода.

Основы Git

Главное отличие Git от любых других СКВ (например, Subversion и ей подобных) – это то, как Git хранит информацию об изменениях данных. Большинство систем хранит информацию как список изменений (патчей, например Δ1, Δ2) для файлов. Эти системы (CVS, Subversion, Perforce, Bazaar) формируют хранимые данные как набор файлов и изменений, сделанных для каждого из этих файлов во времени, как показано на рисунке 5.

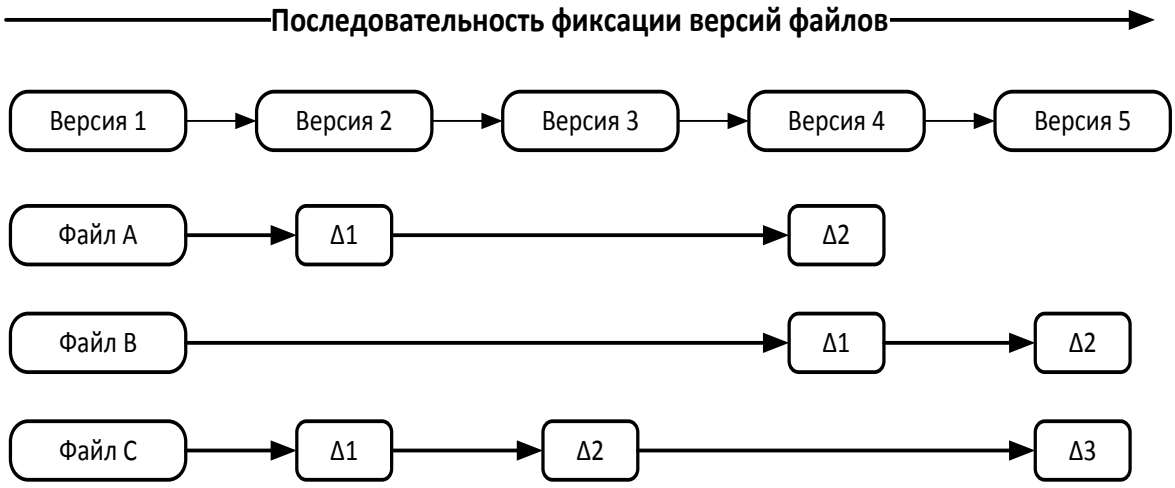


Рисунок 5 *– Хранение данных как изменения к базовой версии для каждого файла*

В Git хранимые данные представляют собой набор слепков небольшой файловой системы. При фиксации текущей версии проекта, Git сохраняет все файлы проекта на текущий момент. Если файл не менялся, то в Git он не сохраняется заново, а делается ссылка на ранее сохранённый файл (рис. 6).

Это важное отличие Git'a от практически всех других систем контроля версий. Git больше похож на небольшую файловую систему с мощными инструментами, работающими поверх неё.

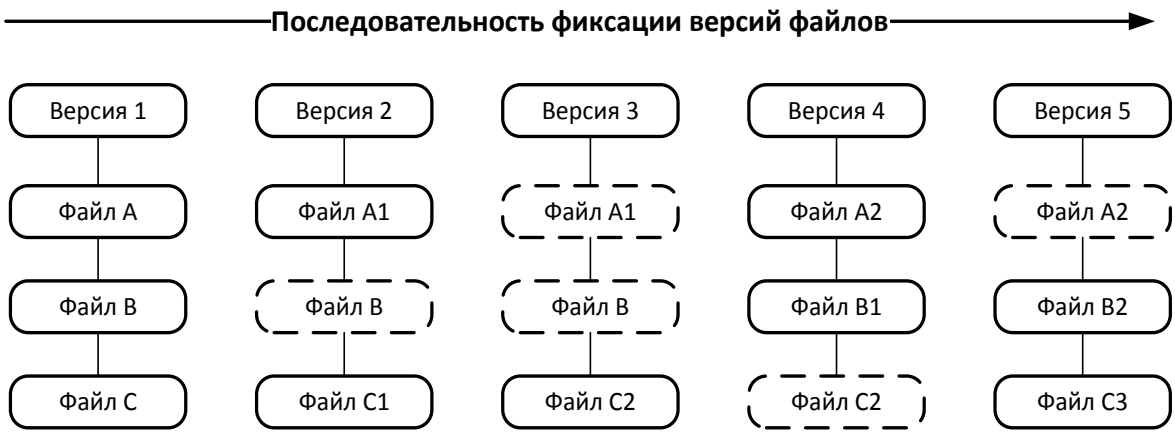


Рисунок 6 *– Git хранит данные как слепки состояний проекта во времени*

Для совершения большинства операций в Git необходимы только локальные файлы и ресурсы. К примеру, чтобы показать историю проекта, Git не нужно скачивать её с сервера, он просто читает её прямо из локального репозитория. Поэтому историю вы увидите практически мгновенно. Если вам нужно просмотреть изменения между текущей версией файла и версией, сделанной месяц назад, Git может взять файл месячной давности и вычислить разницу в локальном репозитории, вместо того чтобы запрашивать разницу у СКВ-сервера или качать с него старую версию файла и делать локальное сравнение.

Кроме того, работа локально означает, что мало чего нельзя сделать без доступа к серверу. Пользователь может работать локально (of-line), фиксируя изменения в локальном репозитории, а при подключении к сети передать изменения в центральный репозиторий на сервере.

Перед сохранением любого файла Git вычисляет контрольную сумму, и она становится индексом этого файла. Поэтому невозможно изменить содержимое файла или каталога так, чтобы Git не узнал об этом. Эта функциональность встроена в сам фундамент Git и является важной составляющей его философии. Если информация потеряется при передаче или повредится на диске, Git всегда это выявит.

При фиксации изменений в базу данных Git данные только добавляются, а не изменяются. Очень сложно заставить систему удалить данные или сделать что-то неотменяемое. Можно, как и в

любой другой СКВ, потерять данные, которые вы ещё не сохранили, но как только они зафиксированы, их очень сложно потерять, особенно если вы регулярно отправляете изменения в другой репозиторий.

В Git файлы могут находиться в одном из трёх состояний: зафиксированном, изменённом и подготовленном. «Зафиксированный» значит, что файл уже сохранён в вашей локальной базе. К «изменённым» относятся файлы, которые пользователь изменил, но ещё не зафиксировал эти изменения. «Подготовленные» файлы – это изменённые файлы, отмеченные для включения в следующую фиксацию файлов (commit).

Таким образом, в проектах, использующих Git, есть три части: каталог Git (Git directory), рабочий каталог (working directory) и об-ласть подготовленных файлов (staging area) (рис. 7).

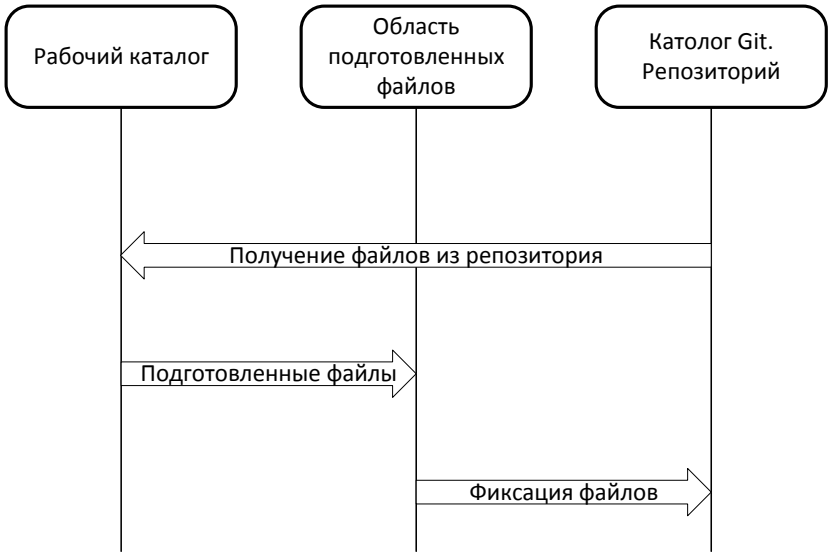


Рисунок 7 *– Рабочий каталог, область подготовленных файлов, каталог Git*

Каталог Git – это место, где Git хранит метаданные и базу данных объектов вашего проекта. Это наиболее важная часть Git, и именно она копируется, когда вы клонируете репозиторий с другого компьютера.

Рабочий каталог – это извлечённая из базы копия определённой версии проекта. Эти файлы достаются из сжатой базы данных в каталоге Git и помещаются на диск, для того чтобы вы их просматривали и редактировали.

Область подготовленных файлов – это обычный файл, хранящийся в каталоге Git, который содержит информацию о том, что должно войти в следующую фиксацию файлов. Иногда его называют индексом (index), но в последнее время становится стандартом называть его областью подготовленных файлов (staging area).

Стандартный рабочий процесс с использованием Git выглядит примерно так:

1. Вы вносите изменения в файлы в своём рабочем каталоге.
2. Подготавливаете файлы, добавляя их слепки в область подготовленных файлов.
3. Делаете фиксацию файлов, которая берёт подготовленные файлы из индекса и помещает их в каталог Git на постоянное хранение.

Если рабочая версия файла совпадает с версией в каталоге Git, файл считается зафиксированным. Если файл изменён, но добавлен в область подготовленных данных, он подготовлен. Если же файл изменился после выгрузки из базы данных, но не был подготовлен, то он считается изменённым.

Главной веткой проекта в Git является ветка master. В процессе разработки можно создавать новые ветки, которые впоследствии могут быть объединены с главной веткой (рис. 8).

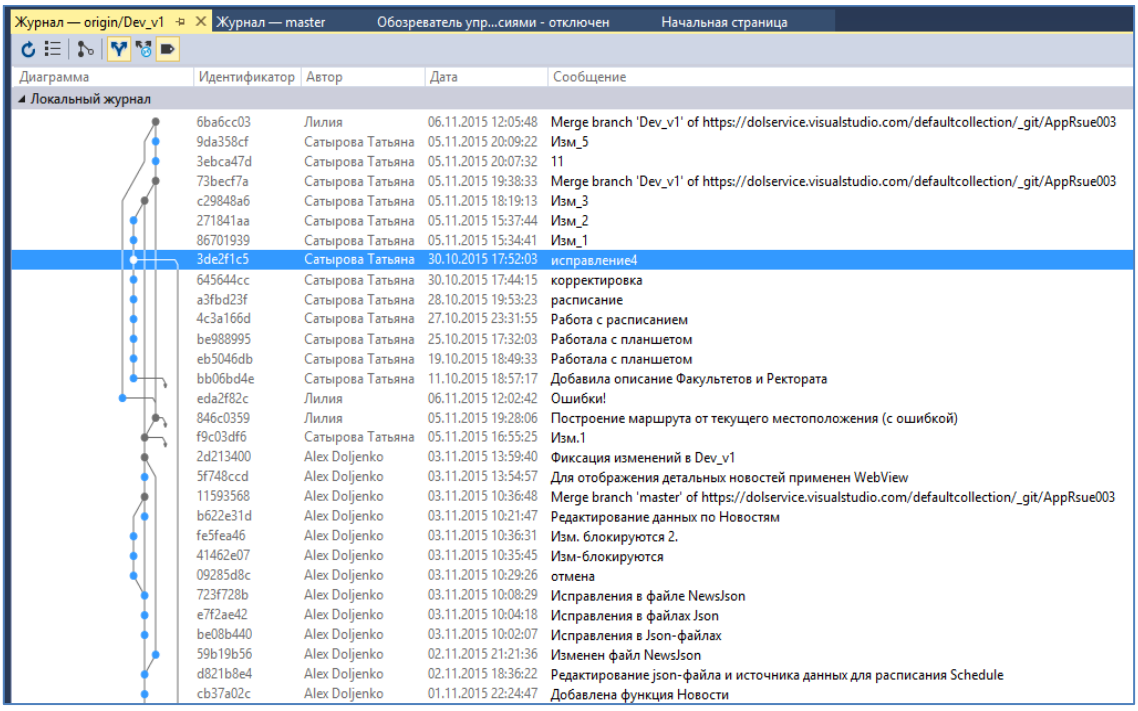


Рисунок 8 *– История проект в Git*

Управление сборками

Процедура компиляции и создания exe, dll файлов по исходникам проекта является важным рабочим процессом проекта разработки программного обеспечения. Данная процедура многократно в день выполняется каждым разработчиком на его собственном компьютере, с его собственной версией проекта. Имеются определенные особенности этого процесса:

- набор подпроектов, собираемых разработчиком; он может собирать не весь проект, а только какую-то его часть; другая часть либо им не используется вовсе, либо не пересобирается очень давно, а по факту она давно изменилась;
- отличаются параметры компиляции.

При этом если не собирать регулярно итоговую версию проекта, то общая интеграция может выявить много разных проблем:

- несоответствие друг другу различных частей проекта;
- наличие специфических ошибок, возникших из-за того, что отдельные проекты разрабатывались без учета параметров компиляции (в частности, переход в Visual Studio с debug на release версию часто сопровождается появлением многочисленных проблем).

Процедуры у сборки проекта часто автоматизируют и выполняют не из среды разработки, а из специального скрипта – build-скрипта. Этот скрипт используется тогда, когда разработчику требуется полная сборка всего проекта. А также он используется в процедуре *непрерывной интеграции* (continues integration), то есть регулярной сборке всего проекта (как правило – каждую ночь). Как правило, процедура непрерывной интеграции включает в себя и регрессионное тестирование, и часто – создание инсталляционных пакетов. Общая схема автоматизированной сборки представлена на рис. 9.

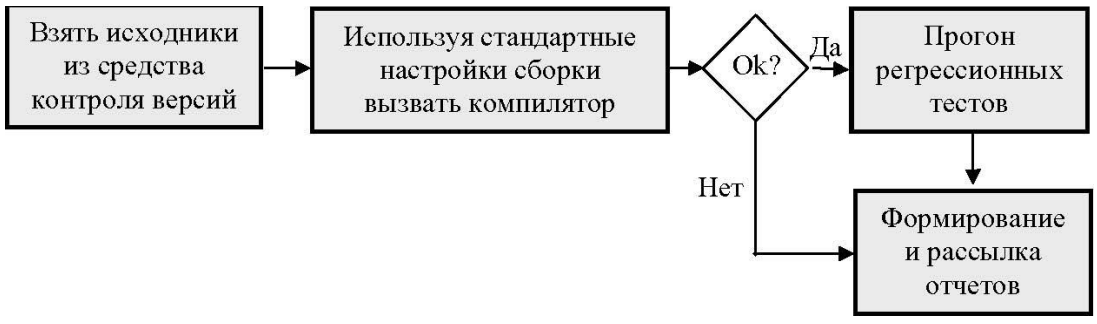


Рисунок 9 *– Общая схема автоматизированной сборки*

Тестировщики должны тестировать по возможности итоговую и целостную версию продукта, так что результаты регулярной сборки оказываются очень востребованы. Кроме того, наличие базовой, актуальной, целостной версии продукта позволяет организовать разработку в итеративно-инкрементальном стиле, то есть на основе внесения изменений. Такой стиль разработки называется baseline-метод.

Понятие baseline

Baseline – это базовая, последняя целостная версия некоторого продукта разработки, например, документации, программного кода и т.д. Подразумевается, что разработка идет не сплошным потоком, а с фиксацией промежуточных результатов в виде текущей официальной версии разрабатываемого актива. Принятие такой версии сопровождается дополнительными действиями по оформлению, сглаживанию, тестированию, включению только законченных фрагментов и т.д. Это результат можно посмотреть, отдать тестировщику, передать заказчику и т.д. Baseline служит хорошим средством синхронизации групповой работы.

Baseline может быть совсем простой – веткой в средстве управления версиями, где разработчики хранят текущую версию своих исходных кодов. Единственным требованием в этом случае может быть лишь общая компилируемость проекта.

Baseline может также поддерживаться непрерывной интеграцией. Важно, что Baseline (особенно в случае с программными активами) не должна устанавливаться слишком рано. Сначала нужно написать какое-то количество кода, чтобы было что интегрировать. Кроме того, вначале много внимания уделяется разработке основных архитектурных решений, и целостная версия оказывается невостребованной. Но начиная с какого-то момента она просто необходима. Какой этот момент – решать членам команды. Наконец, существуют проекты, где автоматическая сборка не нужна вовсе – это простые проекты, разрабатываемые небольшим количеством участников, где нет большого количества исходных текстов программ, проектов, сложных параметров компиляции.

Ключевые термины

Файл – это виртуальная информационная единица, у которой может быть много версий.

Версия файла – отражает текущее его состояние.

Управление версиями – деятельность по сохранению, отслеживанию и получению файлов в процессе разработки программного обеспечения.

Управление сборками – это автоматизированный процесс трансформации исходных текстов ПО в пакет исполняемых модулей.

Система контроля версий – это система, регистрирующая изменения в одном или нескольких файлах, с тем чтобы в дальнейшем была возможность вернуться к определённым старым версиям этих файлов.

Baseline – это базовая, последняя целостная версия файлов некоторого продукта разработки.

Контрольные вопросы

1. Назначение конфигурационного управления программными продуктами.
2. Единицы конфигурационного управления.
3. Основные характеристики единицы конфигурационного управления.
4. Управление версиями файлов.
5. Управления сборками программной системы.
6. Понятие базовой линии (baseline).
7. Управление конфигурацией программной системы.
8. Функции процесса Управления конфигурацией.
9. Управление релизами программного продукта.
10. Классификация релизов по масштабу.
11. Классификация релизов по способу реализации.
12. Функции процесса управления релизами.